

Teórica 7.1

UML – Diagramas de pacotes

Porque é que o pacote de software acabou com a sua dependência?

Diagramas de pacotes

Os diagramas de pacotes mostram a estrutura do sistema projectado, ao nível dos pacotes, agrupando quaisquer elementos do modelo

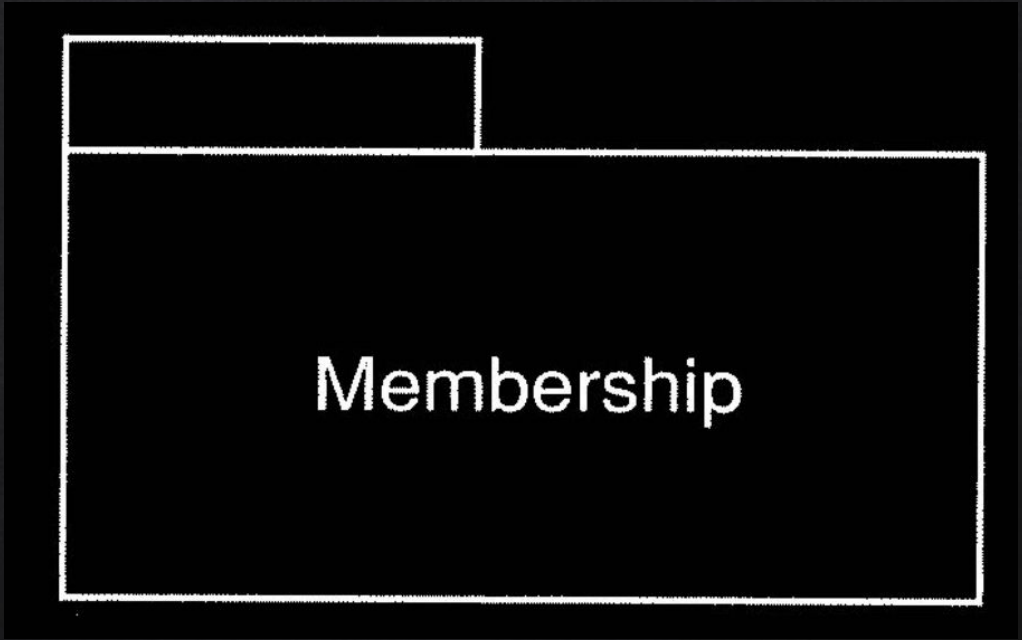
O pacote é o mecanismo UML para agrupar elementos

- Fornece um *namespace* encapsulado dentro do qual todos os nomes devem ser únicos
- Agrupa elementos semanticamente relacionados
- Define um limite semântico no modelo
- Fornece unidades para trabalho paralelo e gestão de configuração
- Os pacotes são um mecanismo de agrupamento lógico

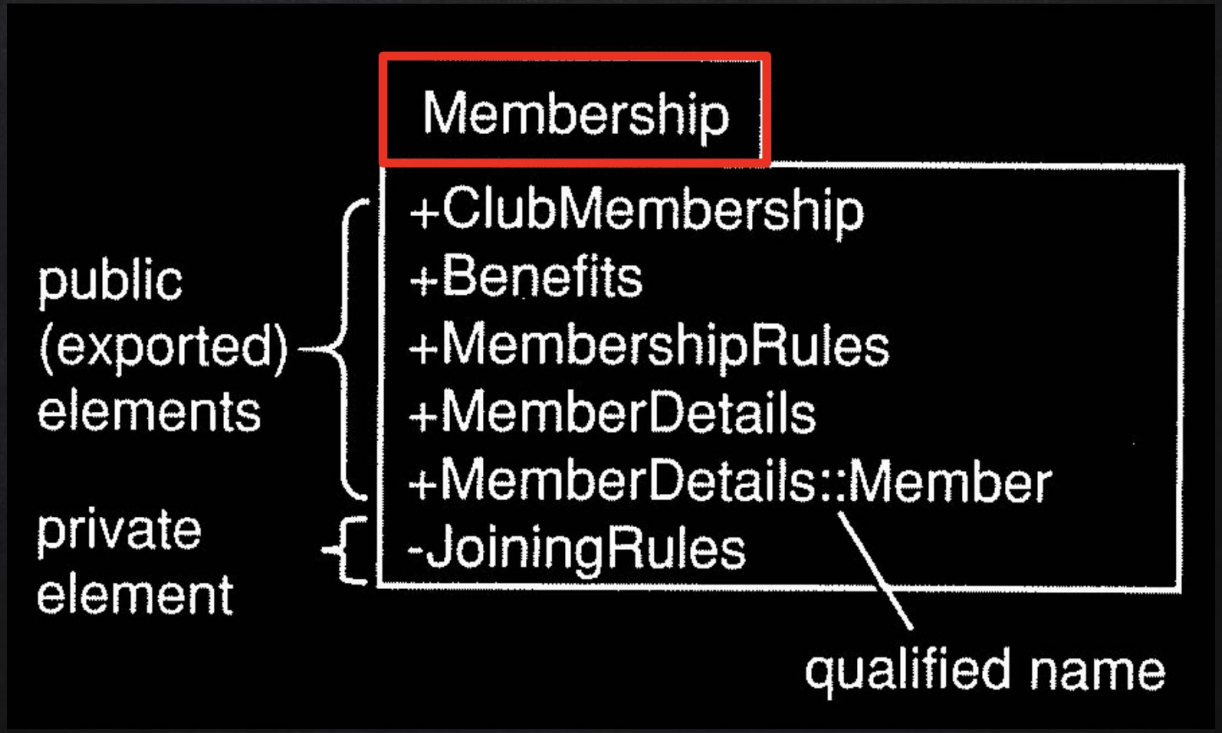
Cada elemento do modelo pertence a um pacote

- A hierarquia de propriedade forma uma árvore
- O pacote de nível superior é estereotipado como <<toplevel>>
- Por definição, todos os elementos do modelo são propriedade do pacote de nível superior
- A hierarquia de pacotes forma uma hierarquia de *namespaces*, onde o pacote de nível superior é a raiz do *namespace*

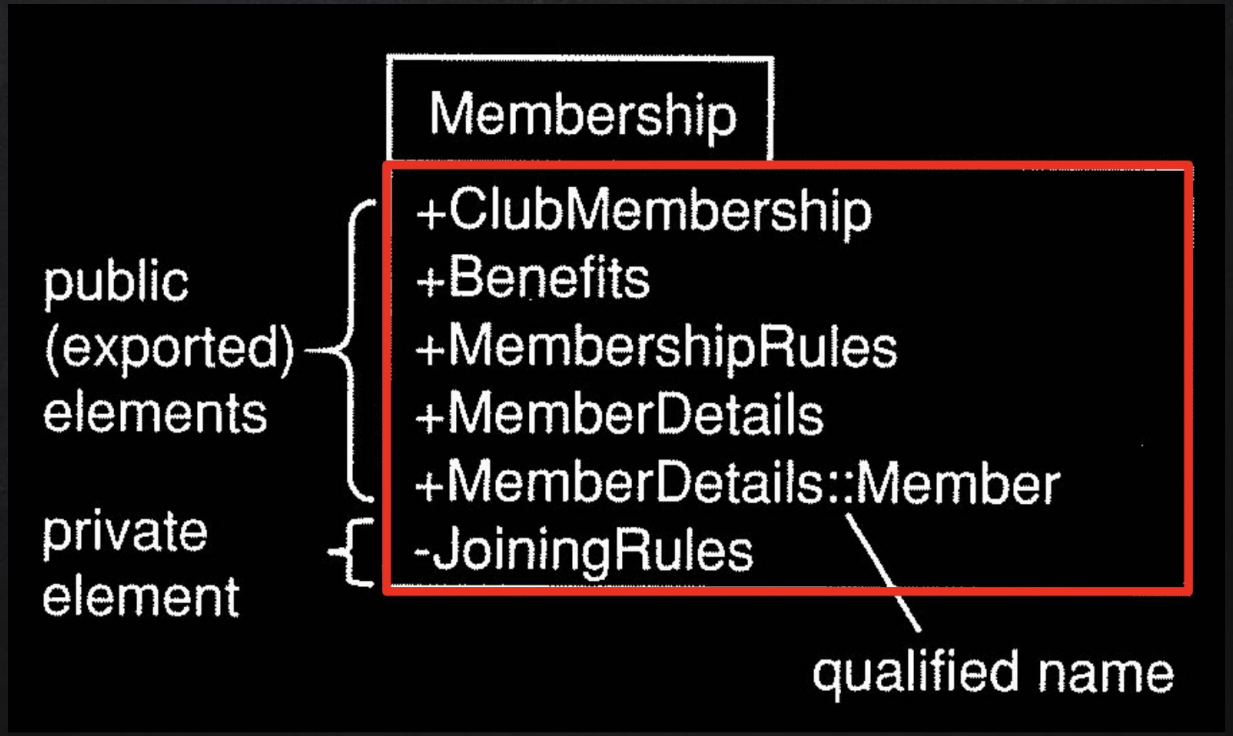
Um pacote UML é representado como uma pasta



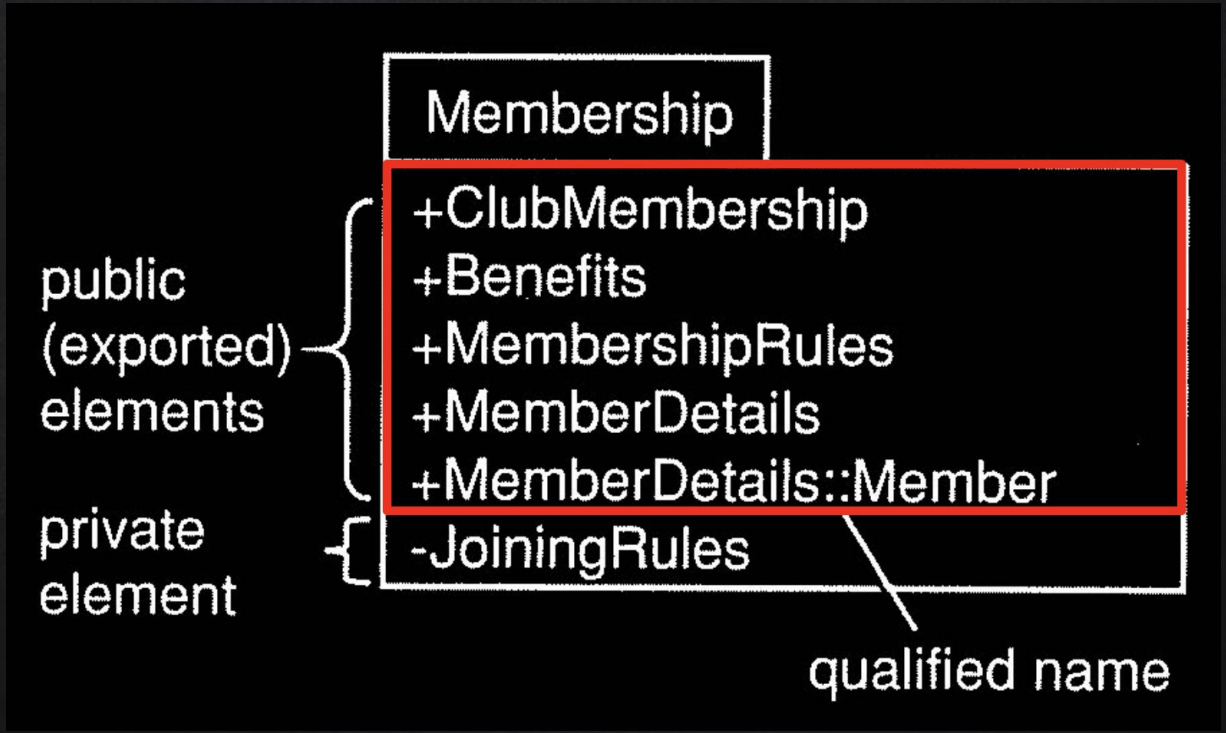
O nome do pacote pode ser representado tab, se o conteúdo do pacote estiver a ser exibido



O nome do pacote pode ser representado na guia, se o conteúdo do pacote estiver a ser



Elementos com visibilidade pública (+) são visíveis para elementos fora do pacote



Elementos com visibilidade privada (−) são visíveis para elementos dentro do pacote



A visibilidade determina se um elemento do pacote é visível fora do pacote ou não

- Usa-se a visibilidade para controlar o grau de acoplamento entre pacotes
- Manter a interface do pacote pequena e simples
  - Minimizar os elementos do pacote com visibilidade pública
  - Maximizar os elementos do pacote com visibilidade privada

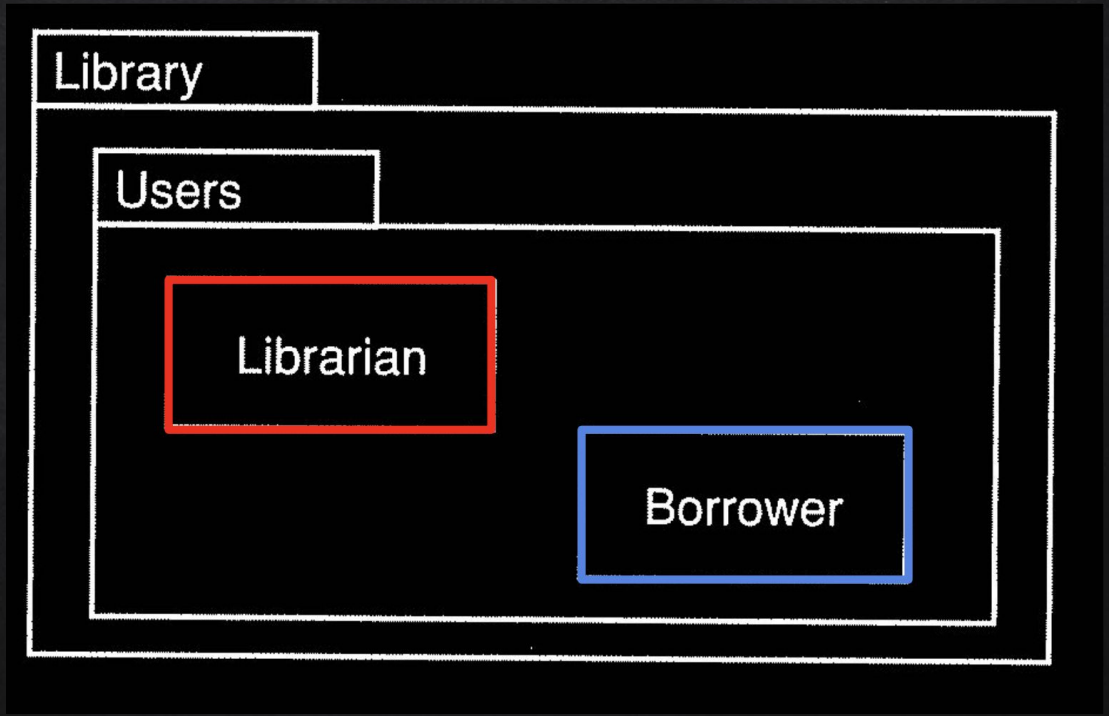
Pacotes e namespaces

- Um pacote define um *namespace* encapsulado
- Dentro de um *namespace*, todos os elementos têm nomes únicos
- Ao referir-se a um elemento de outro *namespace*, precisamos de usar o nome qualificado
- Os nomes qualificados são semelhantes aos nomes em directorias

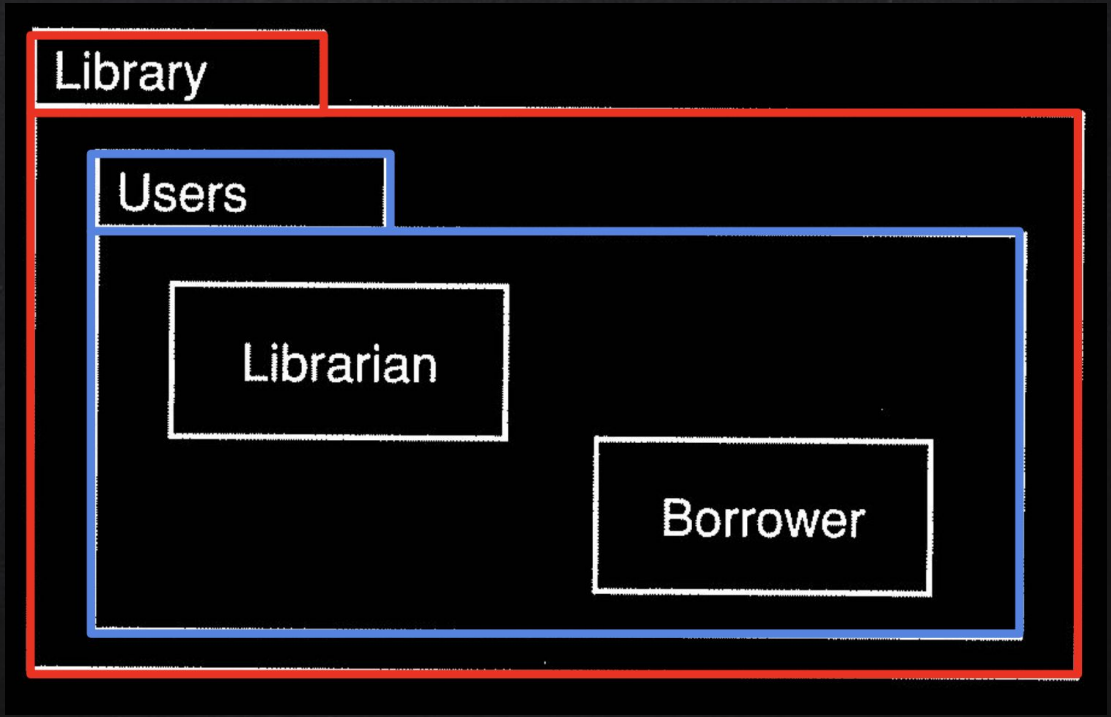
Exemplo: Nomes qualificados de Librarian e Borrower

Library::Users::Librarian

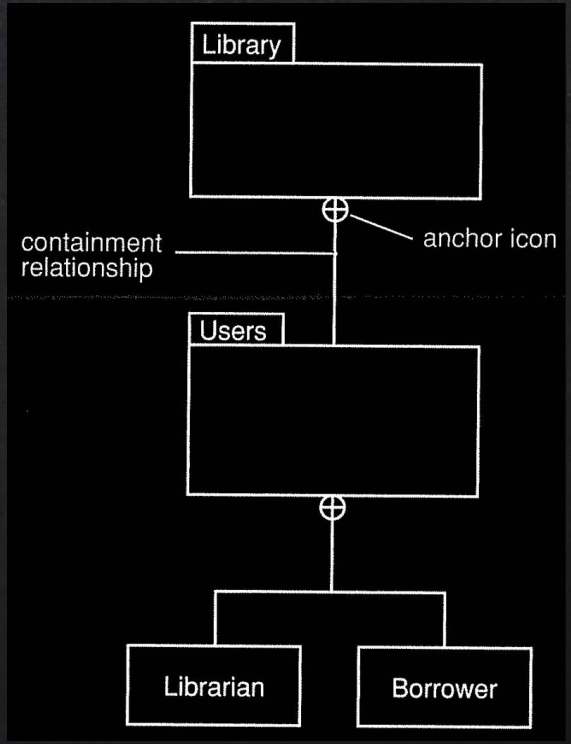
Library::Users::Borrower



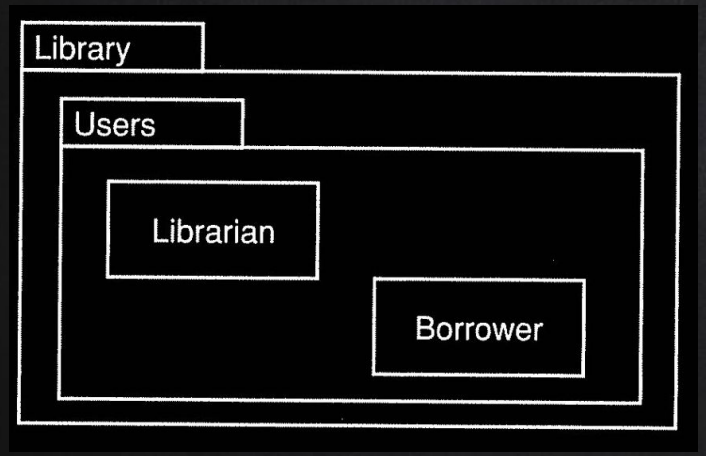
Os pacotes podem ser aninhados em outros pacotes
Users está aninhado em Library

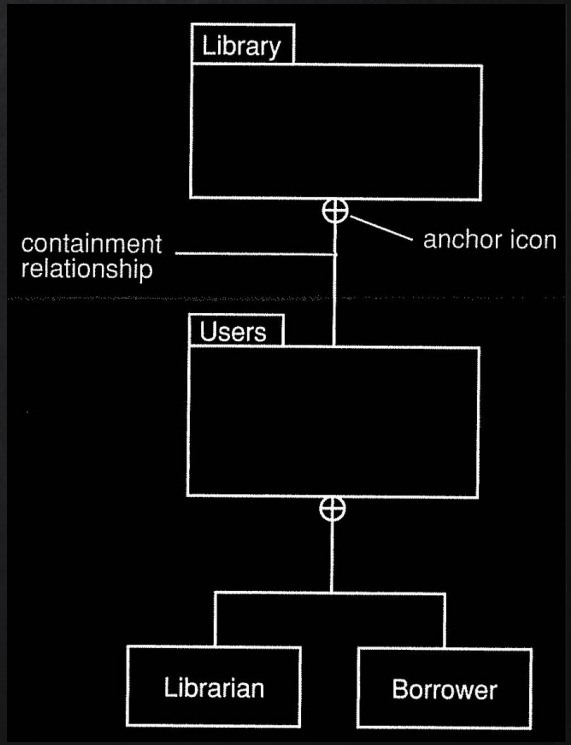


Os pacotes aninhados também podem ser representados com uma sintaxe alternativa



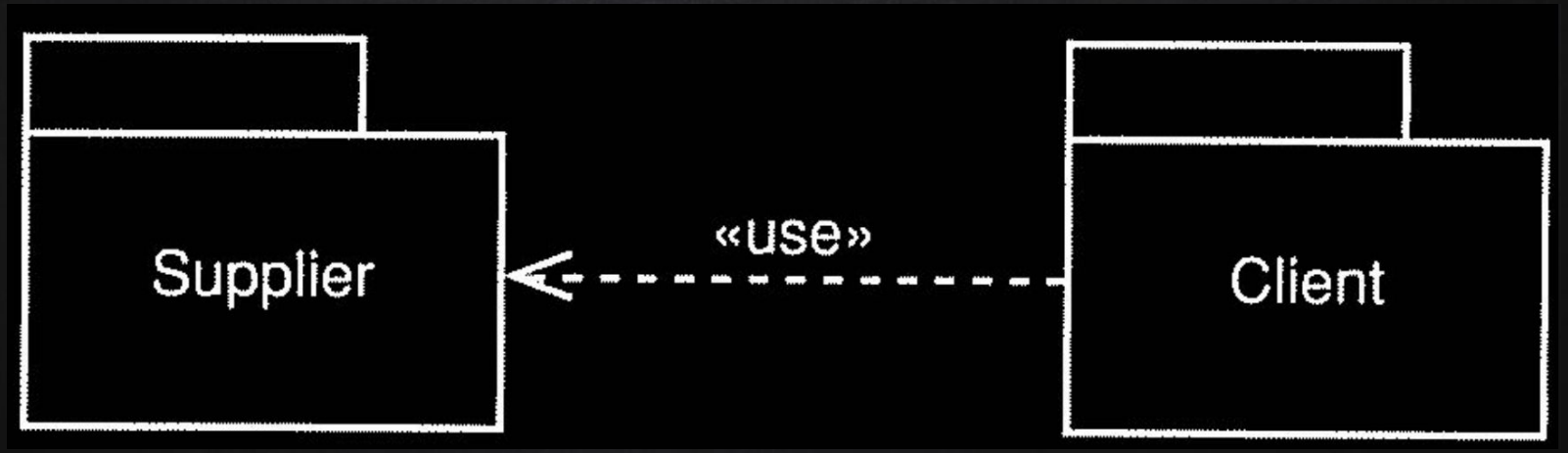
SÃO EQUIVALENTES: UTILIZAR O QUE FOR MAIS EFICAZ PARA A COMUNICAÇÃO EM CADA CASO





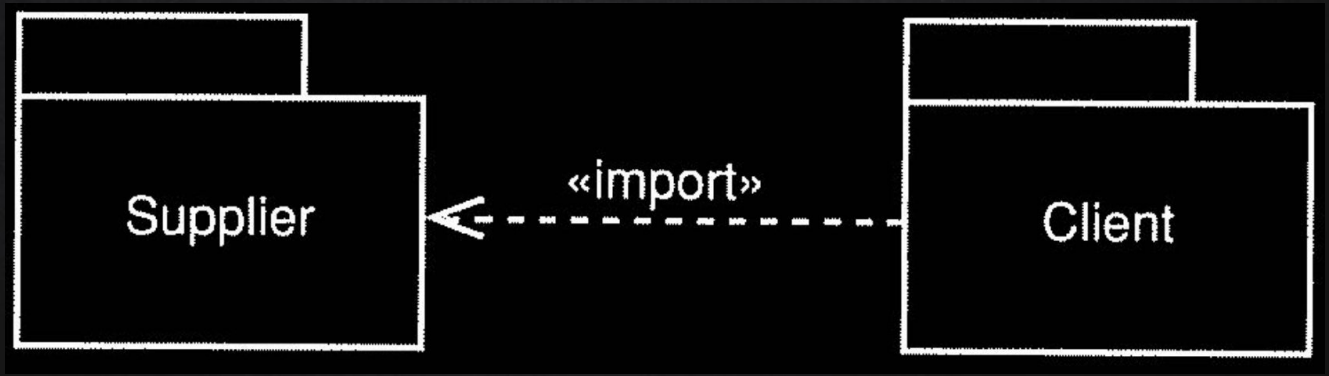
Dependência <<use>>

- Um elemento no pacote de `Client` utiliza um elemento público no pacote do `Supplier` de alguma forma
  - `Client` depende de `Supplier`
- É a dependência padrão



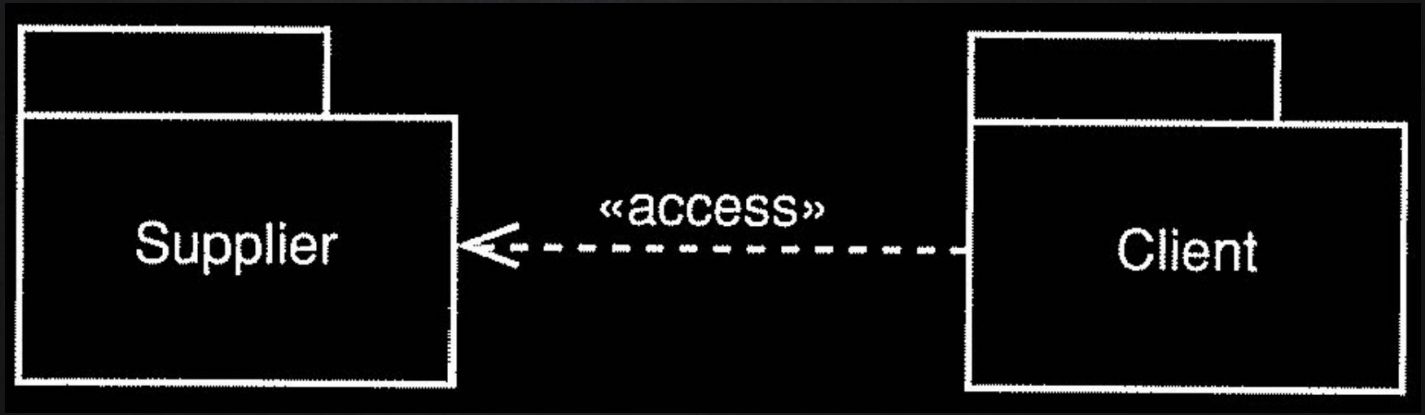
Dependência «import»

- Os elementos públicos do *namespace* de `Supplier` são adicionados como elementos públicos do *namespace* de `Client`
- Os elementos em `Client` podem aceder a todos os elementos públicos em `Supplier` sem nomes qualificados



Dependência «access»

- Os elementos públicos do *namespace* de `Supplier` são adicionados como elementos privados do *namespace* de `Client`
- Os elementos em `Client` podem aceder a todos os elementos públicos em `Supplier` sem nomes qualificados



Transitividade das dependências

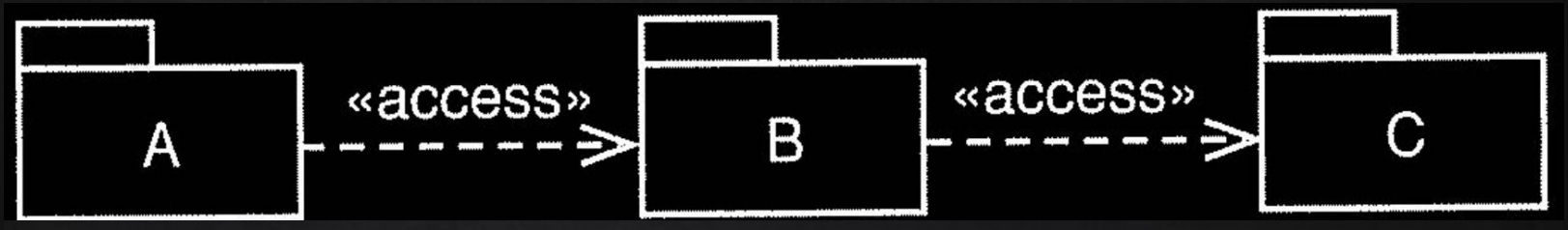
Algumas dependências são transitivas, outras não

- <<import>> é transitiva
  - Elementos *imported* passam a ser públicos
- <<access>> não é transitiva
  - Elementos *accessed* passam a ser privados

A falta de transitividade em «access» permite gerir o acoplamento e a coesão

Nada é acedido, a menos que esteja explícito

- elementos públicos no pacote C tornam-se elementos privados no pacote B
- elementos públicos no pacote B tornam-se elementos privados no pacote A
- elementos no pacote A não têm acesso aos elementos no pacote C



Porque é que o pacote de software acabou com a sua dependência?

Porque não conseguiu lidar com a pressão!



Referências

UML 2 and the Unified Process: Practical Object-Oriented Analysis and Design

2ª Edição, Jim Arlow and Ila Neustadt, Pearson Education

Capítulos 7, 8, 9 e 10

